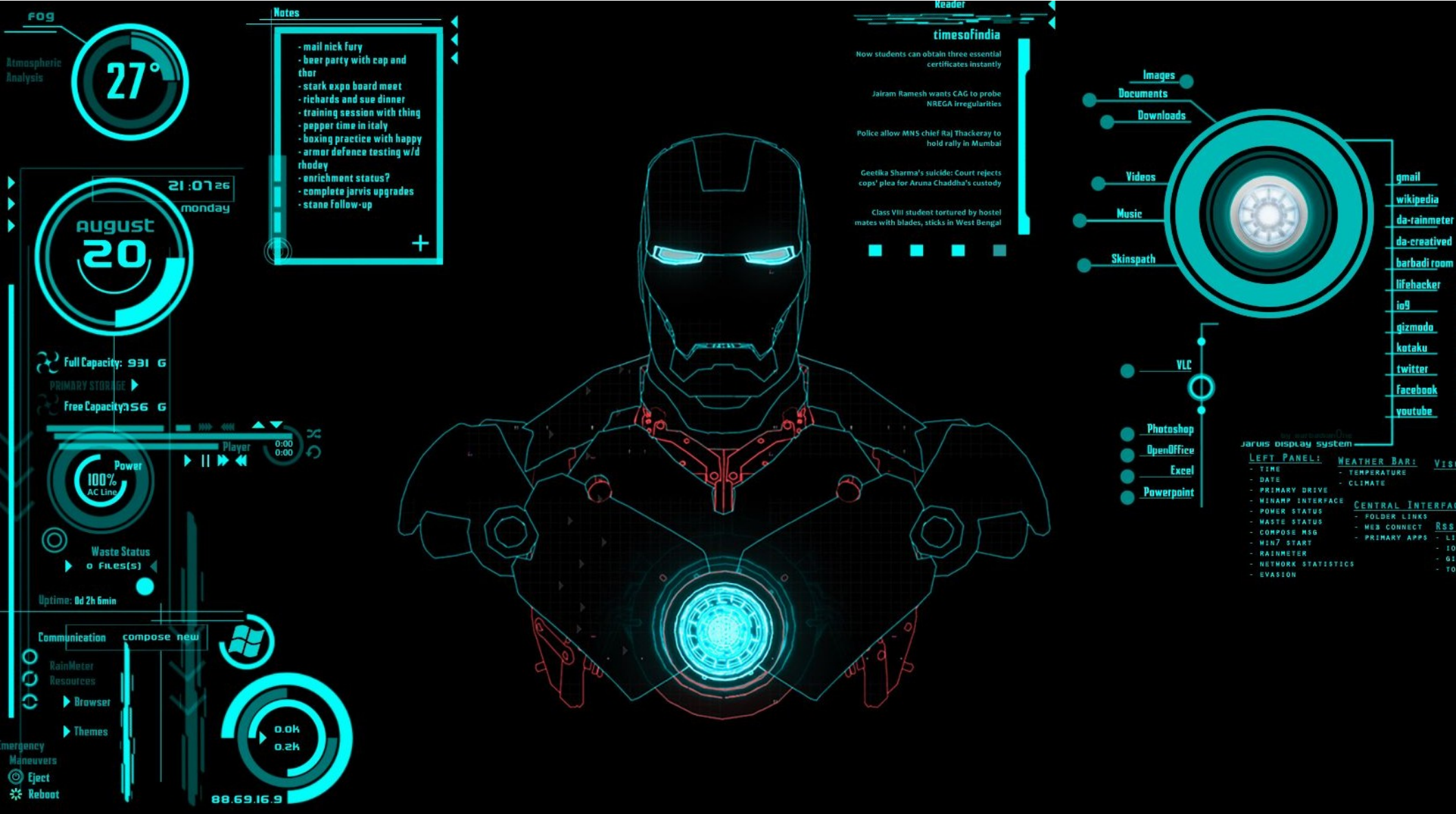




10.OpenGL Shaders

Parte 1

Prof. Alexandre Krohn

Roteiro

- Introdução à Shaders
- Pipeline de Operações
- Programando com Shaders
- Breve introdução à GLSL

Shaders

- ***Shaders*** são programas de computador que transformam **entradas aritméticas** em **saídas gráficas** no dispositivo de visualização

Shaders

- Um **shader** é um programa utilizado para fazer o processamento de imagens, como efeitos especiais, efeitos de cor, de iluminação, e também de sombreamento.
- A posição, brilho, contraste, matiz e outros efeitos em todos os pixels, vértices ou texturas usados para produzir a imagem final na tela podem ser alterados durante o tempo de execução, usando algoritmos construídos nos **shaders**.
- Atualmente, a maioria dos **shaders** são construídos para rodar diretamente na Unidade de Processamento Gráfico (GPU)

Execução em paralelo

- **Shaders** são **executados em paralelo**.
- Isso significa, por exemplo, que um shader pode ser executado uma vez por pixel, com cada uma das execuções rodando simultaneamente em diferentes **threads** na GPU.
- A quantidade de threads simultâneos depende da GPU específica da placa de vídeo, com placas modernas com processadores na casa dos **milhares**.
- Isso tudo significa que os programas de shader podem ter um ótimo desempenho e fornecer aos desenvolvedores muita flexibilidade criativa.

Linguagem de Shaders

- Shaders são descritos em linguagens próprias, semelhantes à linguagem C.
- **Opções disponíveis:**
 - ***C for Graphics (Cg)*** : Desenvolvida pela Nvidia, possui sintaxe quase igual a de C, com a adição de alguns tipos. É suportada pelo ***DirectX*** e pelo ***OpenGL***
 - Estava quase caindo em desuso, mas foi adotada na engine de games ***Unity***, o que causou sua ressurgência.

Linguagem de Shaders

- **High-Level Shading Language (HLSL)** :
Desenvolvida pela **Microsoft** para uso com **DirectX**. Continua em desenvolvimento e uso, e desde a versão **DirectX 12**, suporta alto nível de controle do hardware e grandes ganhos de performance.
- **OpenGL Shading Language (GLSL)** : Criada pelo **OpenGL Architecture Review Board (OpenGL ARB)**. Oferece controle direto do pipeline de execução sem necessitar de linguagem de montagem para acesso ao hardware gráfico. Segue em desenvolvimento.

OpenGL e GLSL

- Como estamos utilizando OpenGL nesta disciplina, veremos um pouco de como funciona o ***pipeline de execução*** da OpenGL e **GLSL**

Estrutura de um programa OpenGL com Shaders

OpenGL Legacy

...

```
glBegin(GL_TRIANGLES);  
glVertex2f(-0.5f,-0.5f);  
glVertex2f(0.0f,0.5f);  
glVertex2f(0.5f,-0.5f);  
glEnd();
```

...

OpenGL Moderno

Program

glCreateProgram



glAttachShader



glAttachShader



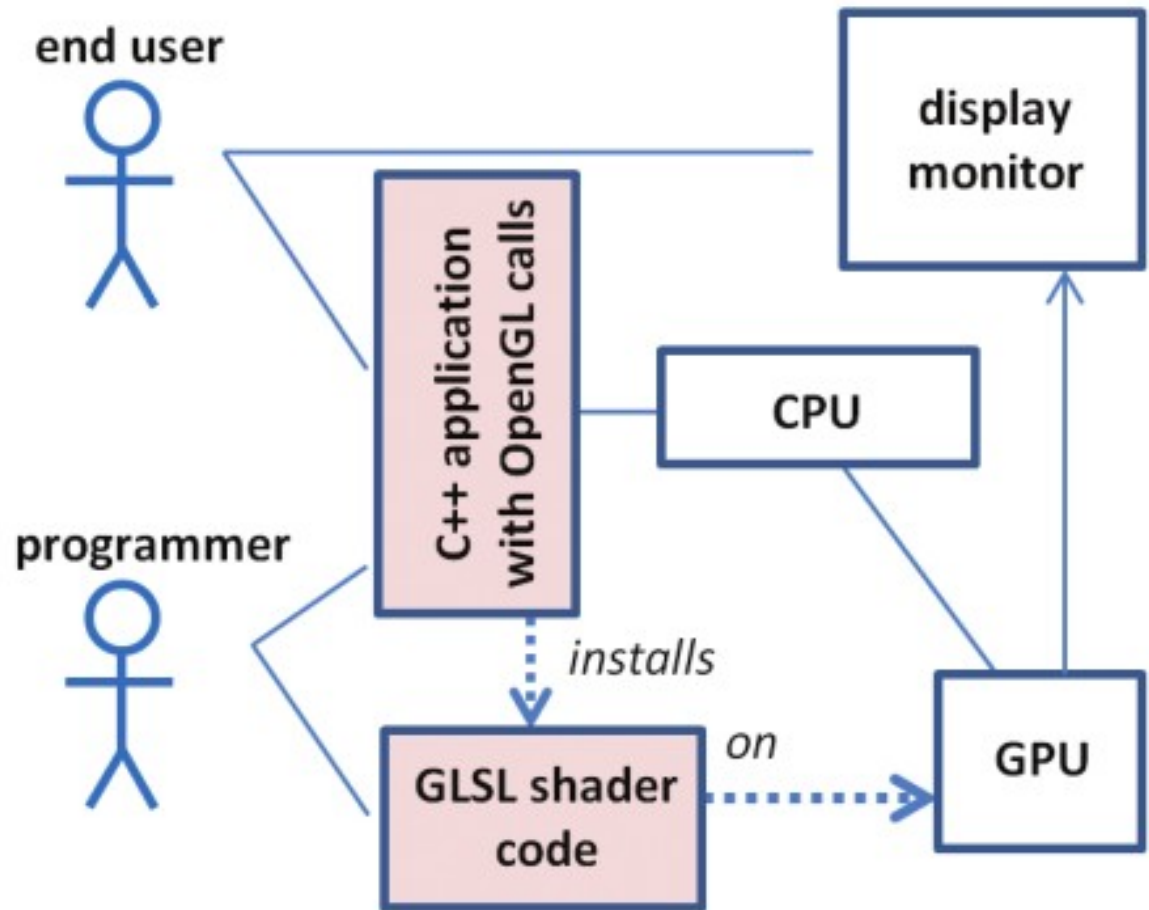
glLinkProgram



glUseProgram

Shaders

- Shaders são programados em separado dos aplicativos gráficos

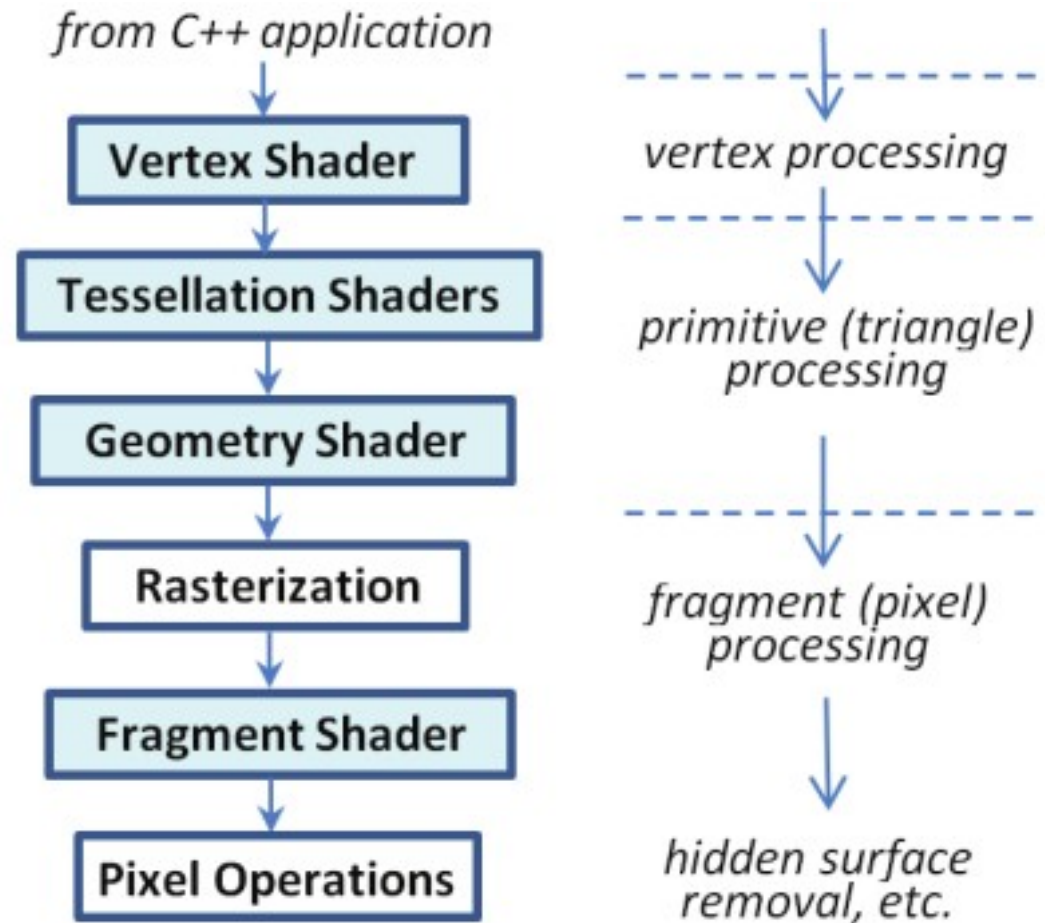


Roteiro

- Introdução à Shaders
- Pipeline de Operações
- Programando com Shaders
- Breve introdução à GLSL

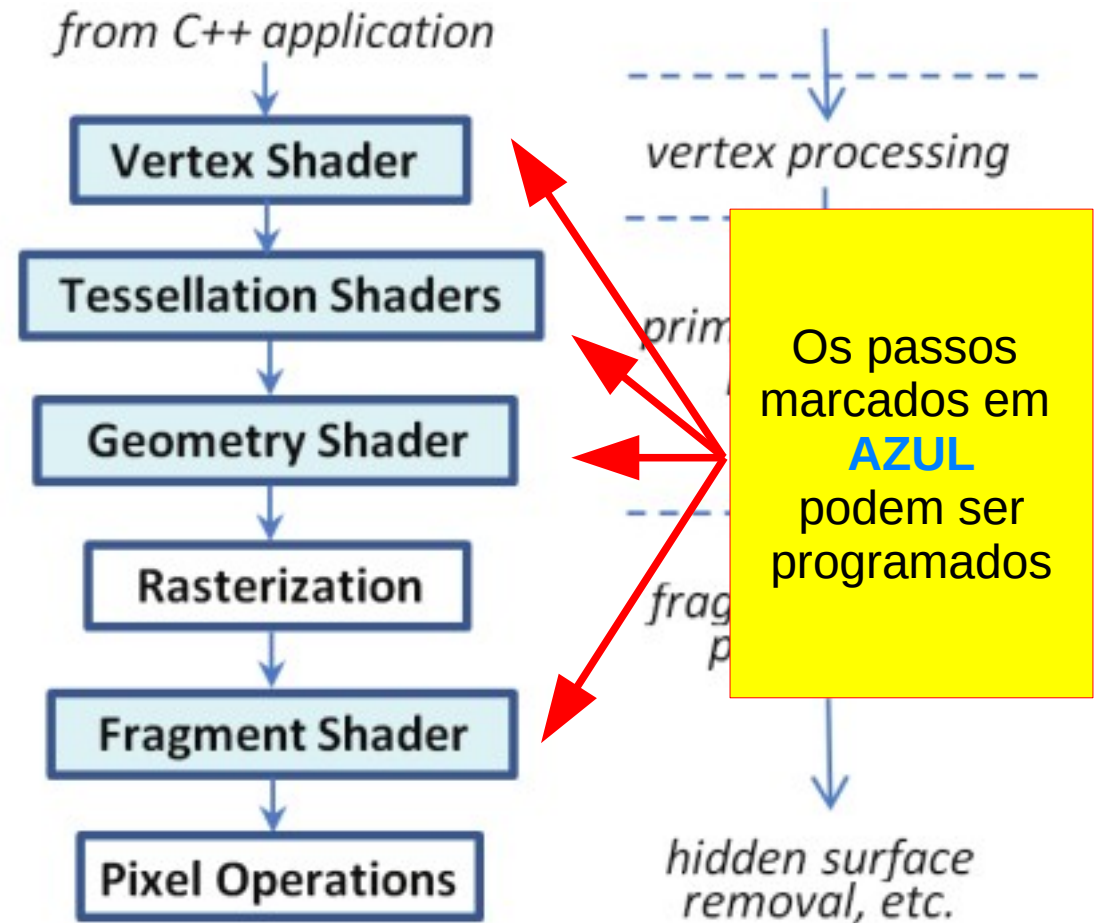
Pipeline de Operações

- A OpenGL moderna utiliza um fluxo de operações chamado de pipeline, para converter uma cena em 3D em uma imagem 2D.

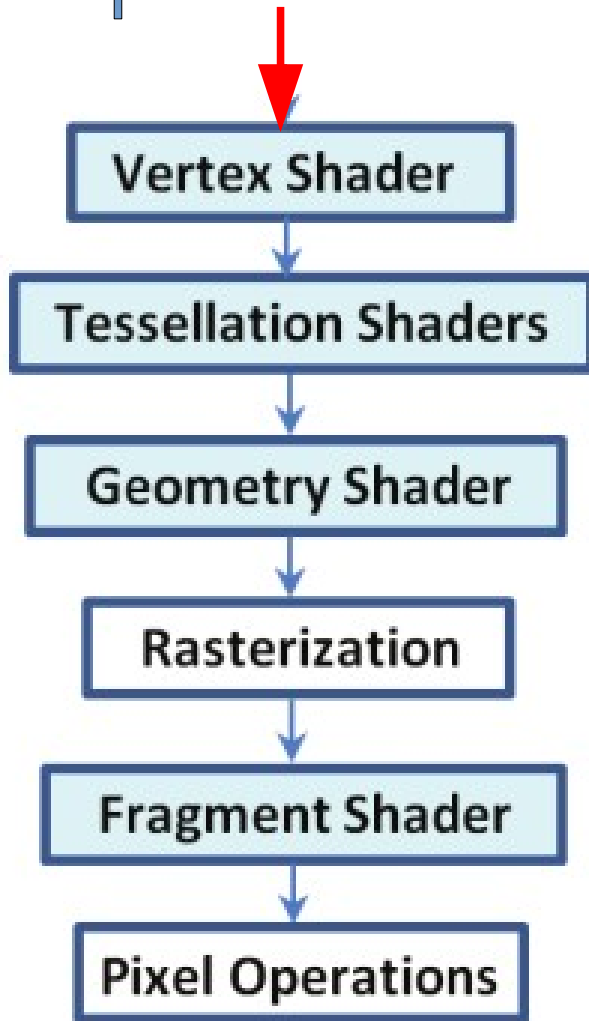


Pipeline de Operações

- A OpenGL moderna utiliza um fluxo de operações chamado de pipeline, para converter uma cena em 3D em uma imagem 2D.

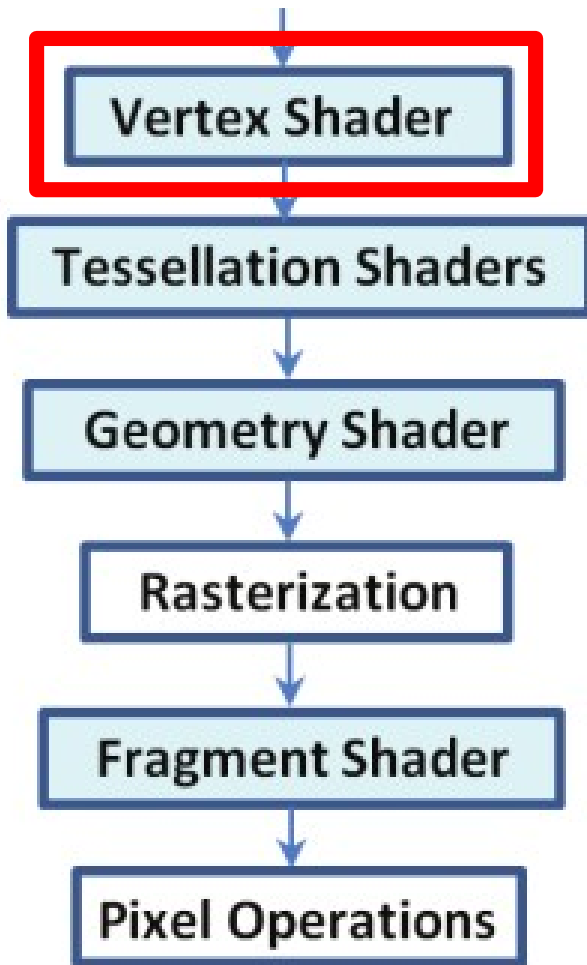


Fluxo de etapas



- A aplicação envia dados de vértices através do pipeline
- Esses dados emergem no dispositivo gráfico na forma de pixels

Vertex Shader



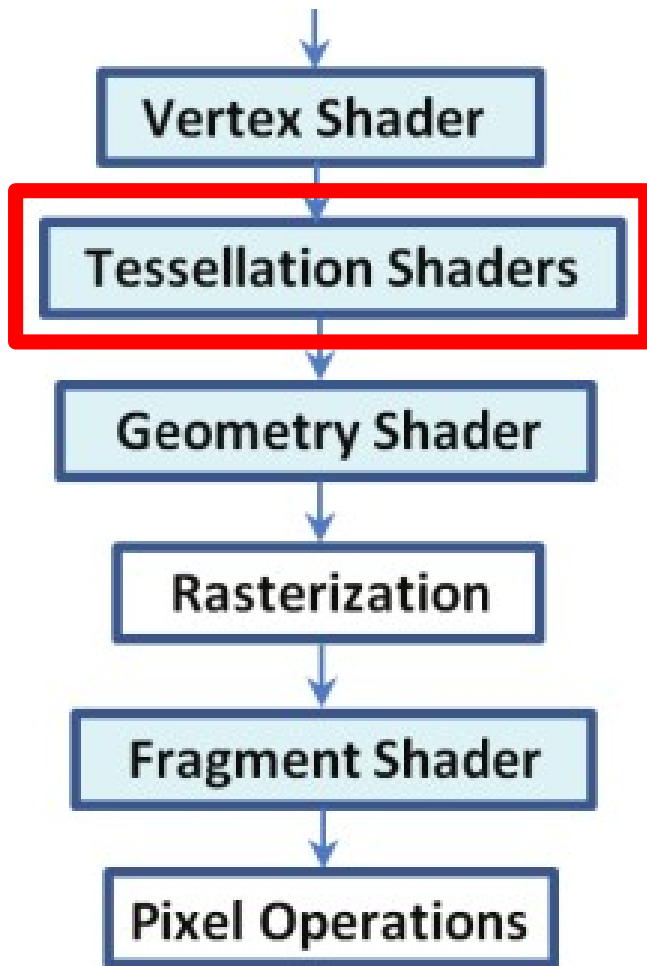
- O vertex shader é o responsável por desenhar os **vértices** do objeto, de acordo com as **primitivas** selecionadas
- É **chamado uma vez para cada vértice**, e pode interpolar os pontos entre os vértices, se necessário, para dar a impressão de continuidade visual.

Vertex Shader

- O vertex shader é executado uma vez por cada vértice. **Muitos vértices : muitas execuções**
- É responsável por :
 - Transformação de **posições**, geralmente utilizando as matrizes de **modelview** e de **projeção**
 - Transformações normais e normalização
 - Geração e transformações de coordenadas de texturas
 - **Iluminação** por vértice
 - **Cálculo** de cores dos vértices

Tessellation Shader

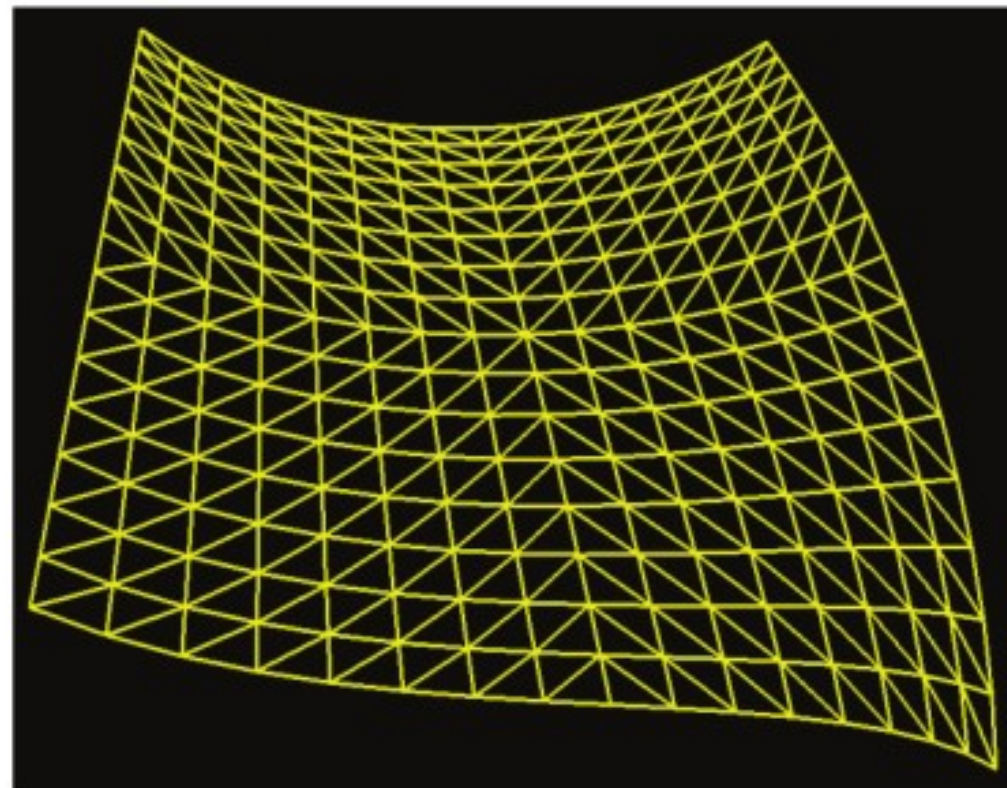
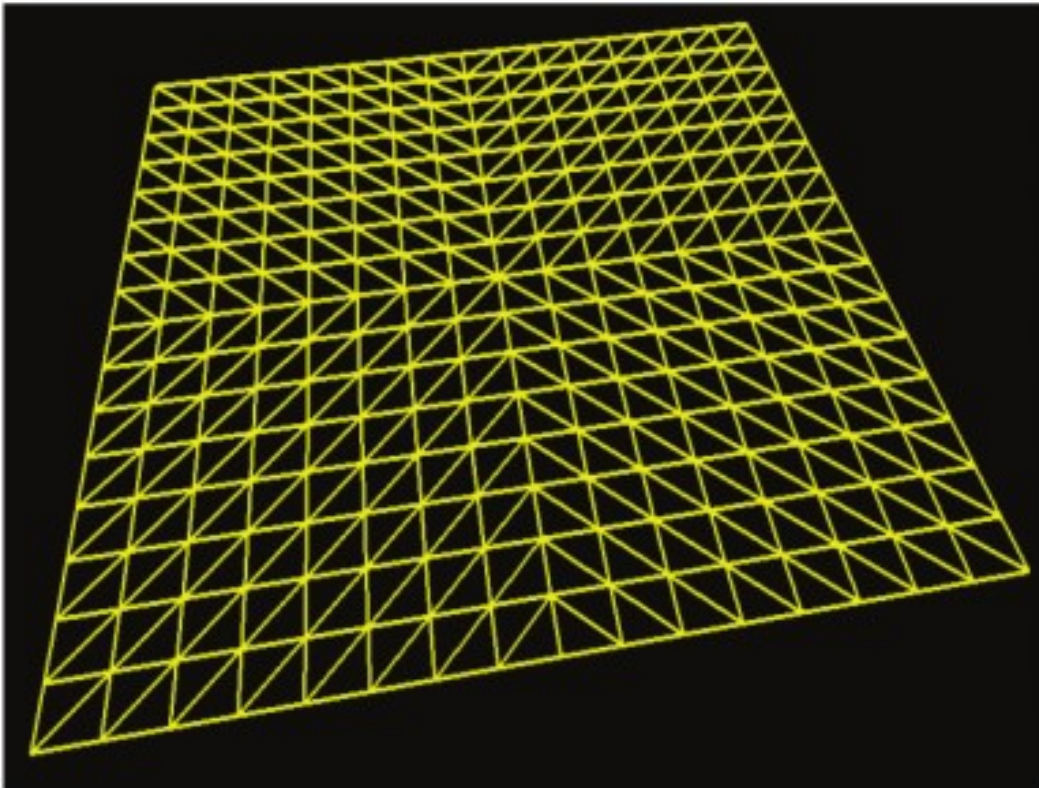
- Tesselelação foi um processo adicionado na OpenGL 4.0



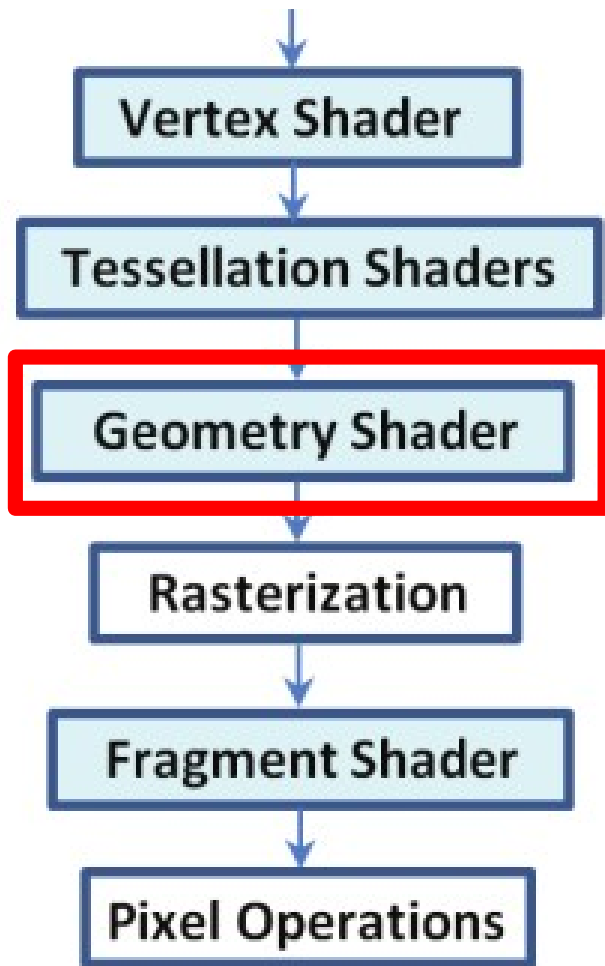
Tessellation Shader

- A **tesselação** é útil quando muitos vértices são necessários para desenhar algo no que, de outra forma, pode ser visto forma simples, como uma área quadrada ou superfície curva
- Também é muito útil para gerando terreno complexo, Nesses casos, é muito mais rápido utilizar a GPU para gerar uma malha de triângulos em hardware do que recorrer à CPU

Tesselação



Geometry Shader

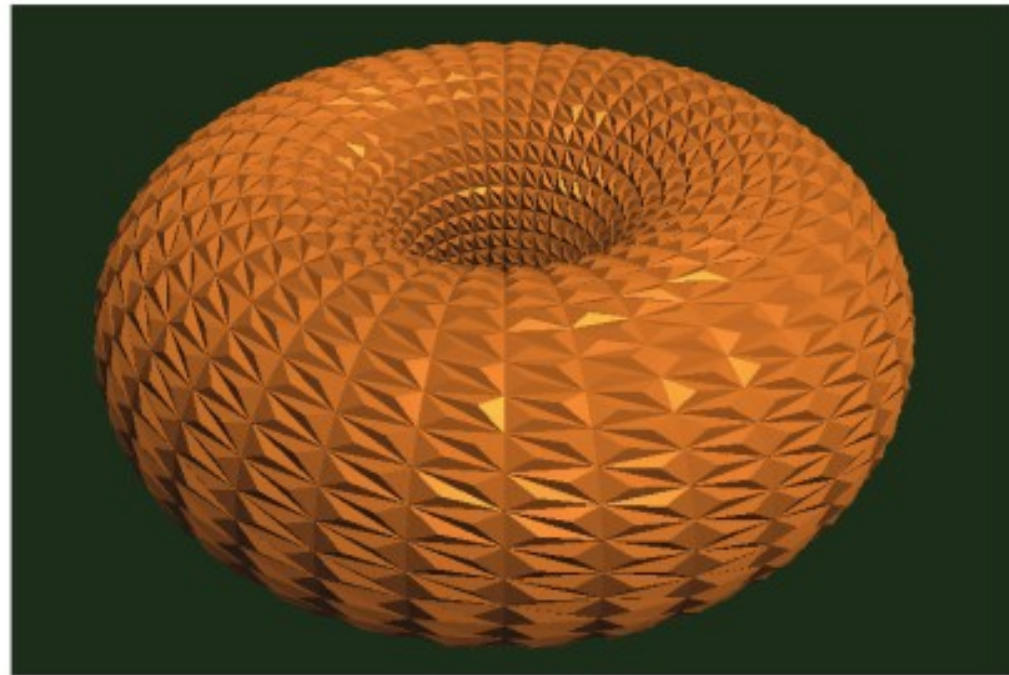
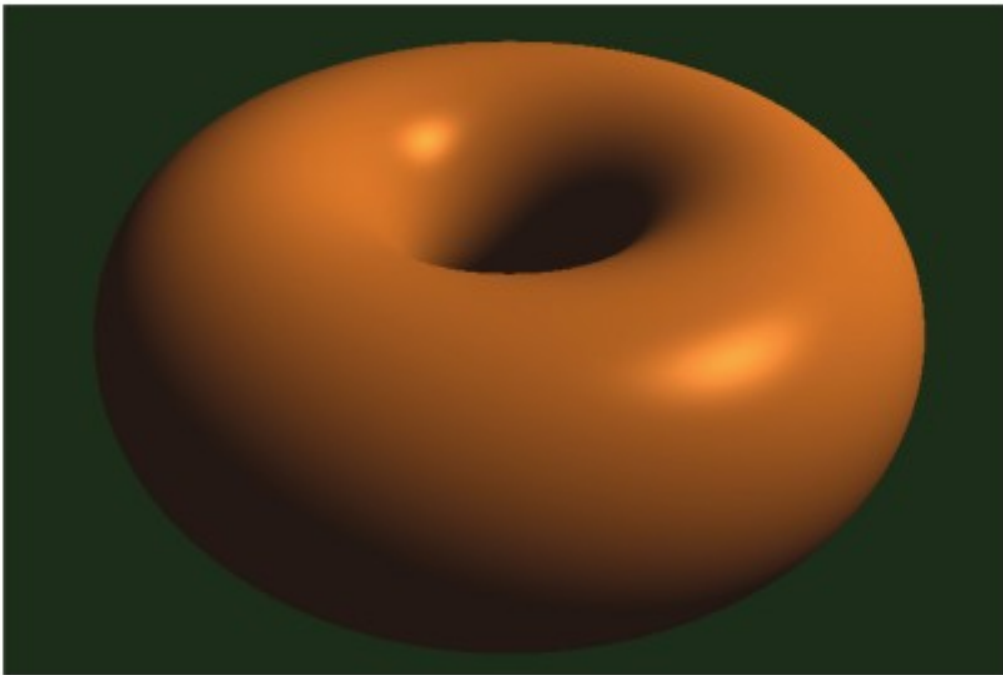


- Enquanto um vertex shader manipula um vértice de cada vez (processamento por vértices),
- O **Geometry Shader** pode **manipular primitivas**.

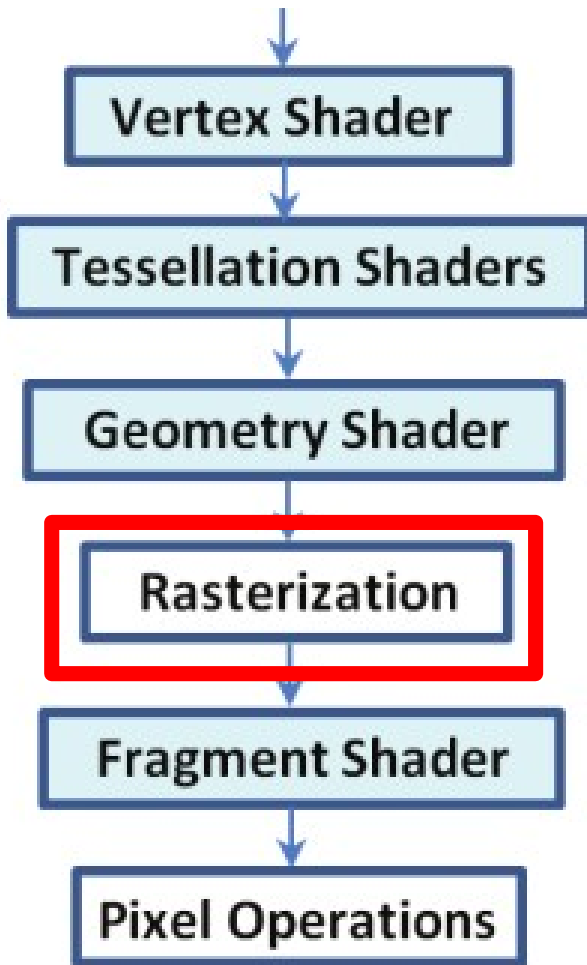
Geometry Shader

- Alterar as primitivas pode ser utilizado para alongá-las, reduzi-las ou provocar “buracos” nas mesmas.
- Pode ser usado para produzir **texturas** como **gramas**, **cabelos** e **pêlos**

Geometry Shaders



Rasterização



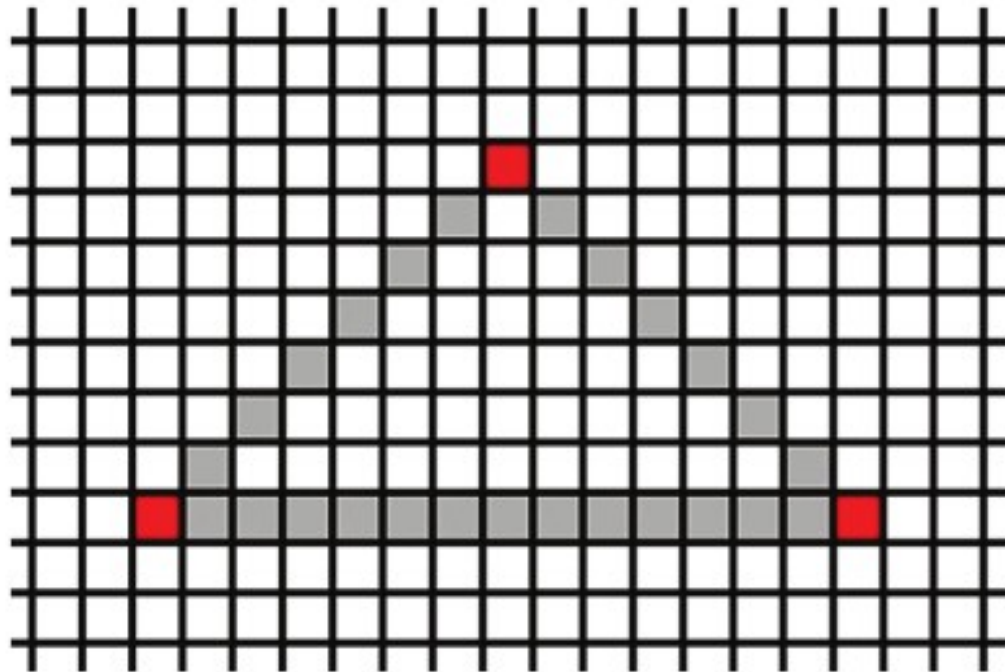
- Primeiro processo do pipeline que não admite programação

Rasterização

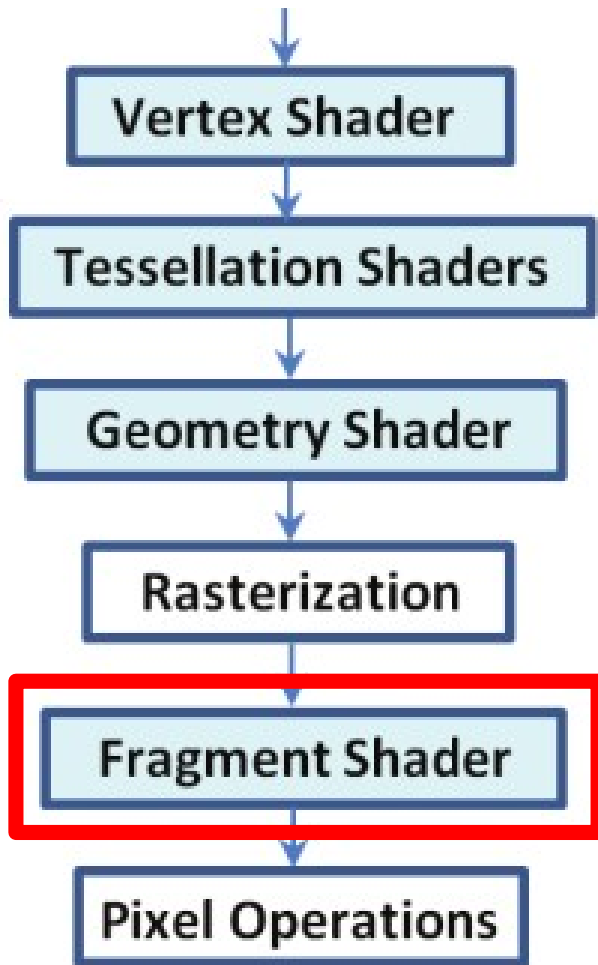
- O objeto 3D composto de vértices, triângulos, cores e outras características precisa ser exibido em um monitor 2D. O monitor contém um ***raster*** - uma matriz retangular de pixels.
- Quando um objeto 3D é rasterizado, o OpenGL converte os primitivos do objeto (geralmente triângulos) em fragmentos.
- Um fragmento contém as informações associadas com um pixel.

Rasterização

- A rasterização determina a localização dos pixels que precisam ser desenhados a fim de produzir o triângulo especificado por seus três vértices.



Fragment Shader



- O ***Fragment Shader*** é o componente responsável por desenhar as ***superfícies*** dos objetos

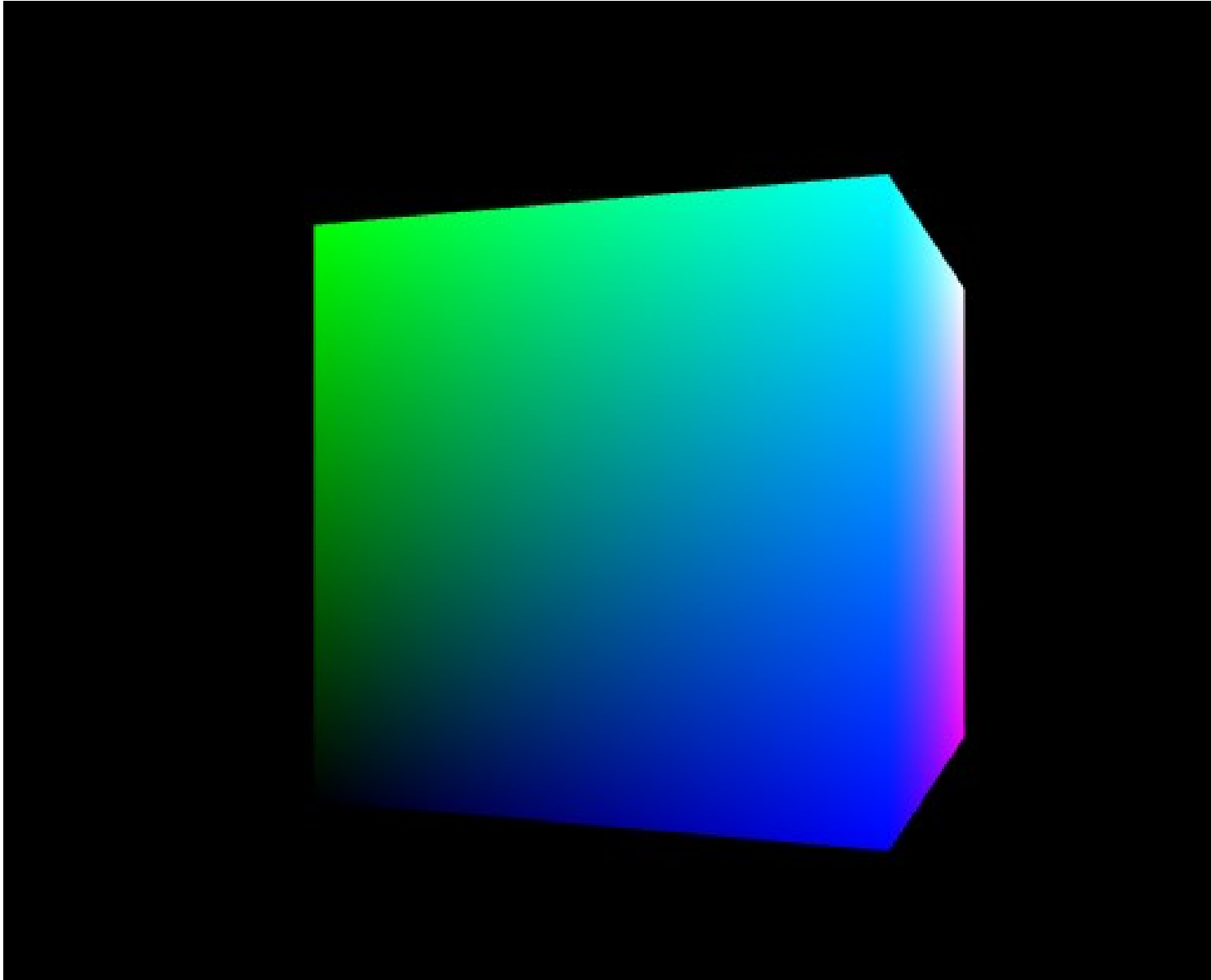
Fragment Shader

- O propósito do **Fragment Shader** é atribuir cores para cada pixel rasterizado
- Por esse motivo ele é também conhecido como *Pixel Shader*.
- O ***Fragment Shader*** é executado *uma vez para cada pixel* exibido no dispositivo gráfico

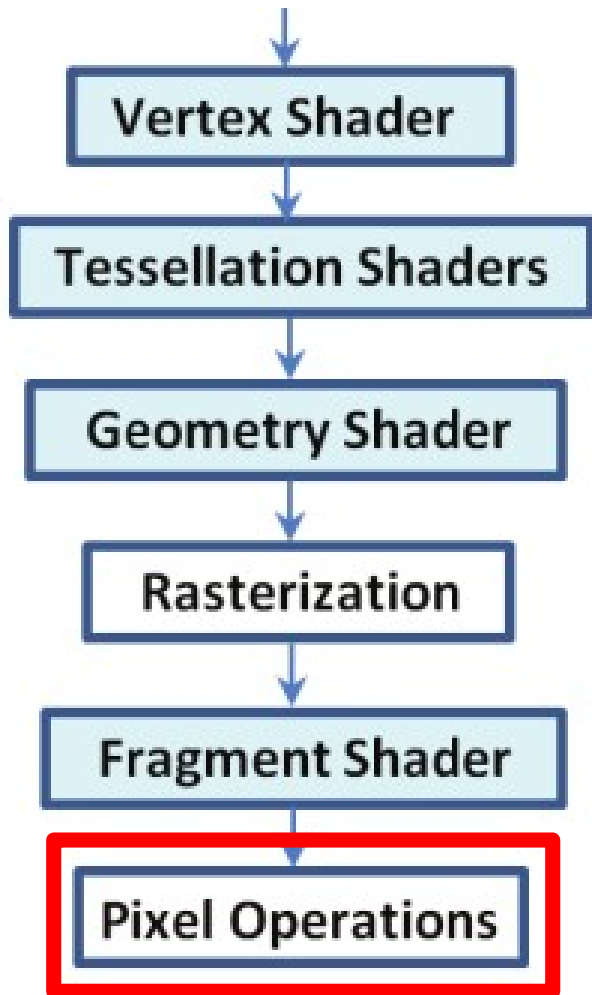
Fragment Shader

- **Calcula a cor de cada pixel da tela** (Executa uma vez para cada pixel)
- Faz o cálculo de cores e coordenadas de textura por pixel
- Aplica **texturas**
- Faz a computação de névoa (esfumaçado), computando normas de **iluminação** por pixel
- Pode descartar o fragmento ou calcular a cor do mesmo

Fragment Shader



Operações de Pixel



- Segundo passo processo do pipeline que não admite programação
- É também o passo final do processo, gerando as imagens

Operações de Pixel

- Uma das operações mais comuns realizadas nessa etapa é a remoção de objetos escondidos (aqueles que ficam por trás dos outros)
- Essa técnica se chama ***Hidden Surface Removal (HSR)***, e é realizada pela OpenGL através o uso de ***Color Buffer*** e de ***Depth Buffer*** (Também conhecido como ***z-Buffer***)

Shaders mais comuns

- Na prática, é necessário fornecer código para os shaders de vértices (***vertex***) e de fragmentos (***fragments***)
- Os passos de tesselação e geometria são opcionais.

Roteiro

- Introdução à Shaders
- Pipeline de Operações
- Programando com Shaders
- Breve introdução à GLSL

Programando com Shaders

- Para poder utilizar OpenGL com Shaders, alguns passos devem ser seguidos:
 - 1) Determinar as posições dos vértices
 - 2) Colocar os vértices na memória
 - 3) Compilar os Shaders
 - 4) Linkar os Shaders
 - 5) Utilizar o programa dos Shaders

```
int main(void)
{
    GLFWwindow* window;

    /* Initialize the library */
    if (!glfwInit())
        return -1;

    /* Create a windowed mode window and its OpenGL context */
    window = glfwCreateWindow(640, 480, "GLFW Básico : Shaders 1", NULL, NULL);
    if (!window)
    {
        glfwTerminate();
        return -1;
    }

    /* Set keyboard events function */
    glfwSetKeyCallback(window, key_callback);

    /* Make the window's context current */
    glfwMakeContextCurrent(window);
```

```
if(glewInit()!=GLEW_OK) {
    std::cout << "Ocorreu um erro iniciando GLEW!" << std::endl;
} else {
    std::cout << "GLEW OK!" << std::endl;
    std::cout << glGetString(GL_VERSION) << std::endl;
}
```

```
float positions[6] = {
    -0.5f, -0.5f,
    0.0f, 0.5f,
    0.5f, -0.5f
};
```

Passo 1

O primeiro trecho do código é
Igual ao da aula passada

Atentem para a definição
dos vértices

Passo 2 (Colocar os dados na memória de vídeo)

```
unsigned int buffer;  
glGenBuffers(1, &buffer);
```

Escreve-se na memória de vídeo usando buffers

```
glBindBuffer(GL_ARRAY_BUFFER, buffer);  
glBufferData(GL_ARRAY_BUFFER, 6 * sizeof(float), positions, GL_STATIC_DRAW);
```

```
glEnableVertexAttribArray(0);  
glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, sizeof(float) * 2, 0);
```

Pede-se ao OpenGL que gere um buffer para onde serão enviados dados

Passo 2 (Colocar os dados na memória de vídeo)

```
unsigned int buffer;  
glGenBuffers(1,&buffer);
```

```
glBindBuffer(GL_ARRAY_BUFFER,buffer);  
glBufferData(GL_ARRAY_BUFFER, 6 * sizeof(float), positions, GL_STATIC_DRAW);
```

```
glEnableVertexAttribArray(0);  
glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, sizeof(float) * 2, 0);
```

Em seguida informa-se que aquele buffer será utilizado para conter um vetor de dados

Passo 2 (Colocar os dados na memória de vídeo)

```
unsigned int buffer;  
glGenBuffers(1,&buffer);
```

```
glBindBuffer(GL_ARRAY_BUFFER,buffer);  
glBufferData(GL_ARRAY_BUFFER, 6 * sizeof(float), positions, GL_STATIC_DRAW);
```

```
glEnableVertexAttribArray(0);  
glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, sizeof(float) * 2, 0);
```

O próximo passo é **vincular o vetor de vértices (**positions**)**
ao buffer

Passo 2 (Colocar os dados na memória de vídeo)

```
unsigned int buffer;  
glGenBuffers(1,&buffer);
```

```
glBindBuffer(GL_ARRAY_BUFFER,buffer);  
glBufferData(GL_ARRAY_BUFFER, 6 * sizeof(float), positions, GL_STATIC_DRAW);
```

```
glEnableVertexAttribArray(0);  
glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, sizeof(float) * 2, 0);
```

Atenção para esse passo : $6 * \text{sizeof}(\text{float})$ significa:

6 é o número de elementos contidos no vetor de vértices
(3 vértices, cada um com as coordenadas x e y ($3 \times 2 = 6$))

$\text{sizeof}(\text{float})$ deve ser usado porque o tipo de dados do vetor de vértices é **float**

Passo 2 (Colocar os dados na memória de vídeo)

```
unsigned int buffer;  
glGenBuffers(1,&buffer);
```

```
glBindBuffer(GL_ARRAY_BUFFER,buffer);  
glBufferData(GL_ARRAY_BUFFER, 6 * sizeof(float), positions, GL_STATIC_DRAW);
```

```
glEnableVertexAttribArray(0);  
glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, sizeof(float) * 2, 0);
```

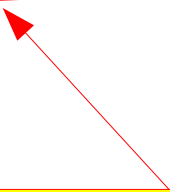
O vetor de atributos deve ser **habilitado**, senão nada será desenhado

Passo 2 (Colocar os dados na memória de vídeo)

```
unsigned int buffer;  
glGenBuffers(1,&buffer);
```

```
glBindBuffer(GL_ARRAY_BUFFER,buffer);  
glBufferData(GL_ARRAY_BUFFER, 6 * sizeof(float), positions, GL_STATIC_DRAW);
```

```
glEnableVertexAttribArray(0);  
glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, sizeof(float) * 2, 0);
```



Define um vetor de ponteiros para indicar como se dá a leitura do vetor de vértices.

Passo 2 (Colocar os dados na memória de vídeo)

```
unsigned int buffer;  
glGenBuffers(1,&buffer);
```

```
glBindBuffer(GL_ARRAY_BUFFER,buffer);  
glBufferData(GL_ARRAY_BUFFER, 6 * sizeof(float), positions, GL_STATIC_DRAW);
```

```
glEnableVertexAttribArray(0);  
glVertexAttribPointer(0, 2, GL_FLOAT, GL_FALSE, sizeof(float) * 2, 0);
```

Começa na posição 0

São lidos de 2 em 2

O tipo dos dados é float

Os dados não devem ser normalizados

Passos 3 e 4 : Compilar e linkar os Shaders

```
std::string vertexShader =  
    "#version 320 es\n"  
    "\n"  
    "layout(location = 0) in vec4 position;\n"  
    "\n"  
    "void main()\n"  
    "{\n"  
    "    gl_Position = position;\n"  
    "}\n";
```

Os shaders são definidos aqui em formato de Strings

```
std::string fragmentShader =  
    "#version 320 es\n"  
    "\n"  
    "precision mediump float;\n"  
    "\n"  
    "layout(location = 0) out vec4 color;\n"  
    "\n"  
    "void main()\n"  
    "{\n"  
    "    color = vec4(1.0, 0.0, 0.0, 1.0);\n"  
    "}\n";
```

Sempre utilizamos pelo menos dois shaders:

Um shader de vértices e um shader de fragmentos

```
unsigned int shader = createShaders(vertexShader, fragmentShader);
```

Passos 3 e 4 : Compilar e linkar os Shaders

```
std::string vertexShader =  
    "#version 320 es\n"  
    "\n"  
    "layout(location = 0) in vec4 position;\n"  
    "\n"  
    "void main()\n"  
    "{\n"  
    "    gl_Position = position;\n"  
    "}\n";
```

```
std::string fragmentShader =  
    "#version 320 es\n"  
    "\n"  
    "precision mediump float;\n"  
    "\n"  
    "layout(location = 0) out vec4 color;\n"  
    "\n"  
    "void main()\n"  
    "{\n"  
    "    color = vec4(1.0, 0.0, 0.0, 1.0);\n"  
    "}\n";
```

```
unsigned int shader = createShaders(vertexShader, fragmentShader);
```

Computação Gráfica

Ambos são compilados individualmente e linkados na função **createShaders**

Função createShaders

```
static unsigned int createShaders(const std::string& vertexShader,
                                  const std::string& fragmentShader) {

    unsigned int program = glCreateProgram();
    unsigned int vs = compileShader(GL_VERTEX_SHADER, vertexShader);
    unsigned int fs = compileShader(GL_FRAGMENT_SHADER, fragmentShader);

    glAttachShader(program, vs);
    glAttachShader(program, fs);

    glLinkProgram(program);
    glValidateProgram(program);

    glDeleteShader(vs);
    glDeleteShader(fs);

    return program;
}
```

Função compileShaders

```
static unsigned int compileShader(unsigned int type, const std::string& source) {  
  
    unsigned int id = glCreateShader(type);  
    const char* src = source.c_str();  
    glShaderSource(id, 1, &src, nullptr);  
    glCompileShader(id);  
  
    int result;  
    glGetShaderiv(id, GL_COMPILE_STATUS, &result);  
    if(result==GL_FALSE) {  
        int length;  
        glGetShaderiv(id, GL_INFO_LOG_LENGTH, &length);  
        char* message = (char *) alloca(length * sizeof(char));  
        glGetShaderInfoLog(id, length, &length, message);  
        std::cout << "Failed to compile " <<  
            (type == GL_VERTEX_SHADER ? "vertex" : "fragment") << " shader!" <<  
            std::endl;  
        std::cout << message << std::endl;  
        glDeleteShader(id);  
        return 0;  
    }  
    return id;  
}
```

Passo 5 : Usar o Shader

Logo após Compilar e Linkar o programa

```
unsigned int shader = createShaders(vertexShader, fragmentShader);
```

Manda-se executá-lo

```
glUseProgram(shader);
```

Finalizando o programa...

```
/* Loop until the user closes the window */
while (!glfwWindowShouldClose(window))
{
    /* Render here */
    glClear(GL_COLOR_BUFFER_BIT);

    /* Desenha o conteúdo da memória, através dos shaders */
    glDrawArrays(GL_TRIANGLES, 0, 3);

    /* Swap front and back buffers */
    glfwSwapBuffers(window);

    /* Poll for and process events */
    glfwPollEvents();
}

glDeleteProgram(shader);
glfwTerminate();
return 0;
}
```

Neste trecho os shaders
são de fato utilizados

Roteiro

- Introdução à Shaders
- Pipeline de Operações
- Programando com Shaders
- Breve introdução à GLSL

GLSL : OpenGL Shader Language

- Sintaxe:
- Parece com C, mas com algumas importantes diferenças
 - Não possui ponteiros
 - Não possui operadores bit a bit
 - Tipos básicos adicionais, incluindo vetores, matrizes, e samplers. Não há doubles ou strings

GLSL : Tipos

- Básicos
 - float, int, bool, sampler
- Vetores
 - vec2, vec3, vec4 → (vetores de floats)
 - ivec2, ivec3, ivec4 → (vetores de inteiros)
 - bvec2, bvec3, bvec4 → (vetores de booleanos)
- Matrizes
 - mat2, mat3, mat4

Operações e facilitadores

- Construtores estilo C++
 - `vec4(1.0, 2.0, 3.0, 4.0)`
 - `vec4.x` => 1.0
 - `vec4.xy` => `vec2(1.0, 2.0)`
 - `vec4.rgba` => `vec4(1.0, 2.0, 3.0, 4.0)`
 - `vec4.b` => 3.0
 - `vec4.xgb` => erro!
- Swizzling e Smearing
 - `vec4.wzyx` => `vec4.abgr` => (4.0, 3.0, 2.0, 1.0)
 - `vec4.yyy` => `vec3(3.0, 3.0, 3.0)`

Operadores sobrecarregados

- Operadores lógicos e aritméticos padrão do C/C++
- Operadores sobrecarregados para operações de matrizes e vetores
 - `mat4 m;`
 - `vec4 a, b, c;`
 - `b = a*m;`
 - `c = m*a;`

Funções

- Nativas
 - Aritméticas: `sqrt`, `power`, `abs`
 - Trigonométricas: `sin`, `asin`
 - Gráficas: `length`, `reflect`
 - Vetoriais: `dot`
 - E outras...
- User defined

Estruturas de controle

- Iguais as da linguagem C:
 - `if() {...}`
 - `while(){...}`
 - `for(...;...;...) { }`

Qualificadores

- **in**

- Usado para indicar valores que passam pelo pipeline de execução do **vertex shader para o fragment shader**

- **out**

- Para dados serão enviados pelo fragment shader **para as operações de pixel**

Qualificadores

- **attribute**

- Para dados que mudam com muita frequência, enviados da aplicação para o vertex shader
 - Ex.: posição, norma, etc...
 - **attribute vec4 gl_Vertex;**
 - **attribute vec3 gl_Normal;**

- **uniform**

- Para dados que não mudam com muita frequência, enviados da aplicação para o vertex shader ou para o fragment shader
 - Ex.: posição da luz, unidade de textura, outras constantes
 - **uniform mat4 gl_ModelViewMatrix;**
 - **uniform mat3 gl_NormalMatrix**

Qualificadores

- **varying**

- Para dados **interpolados** passados do **vertex shader para o fragment shader**

- Ex.: coordenadas de texturas, cores dos vértices

- **varying vec4 gl_Color; // fragment**

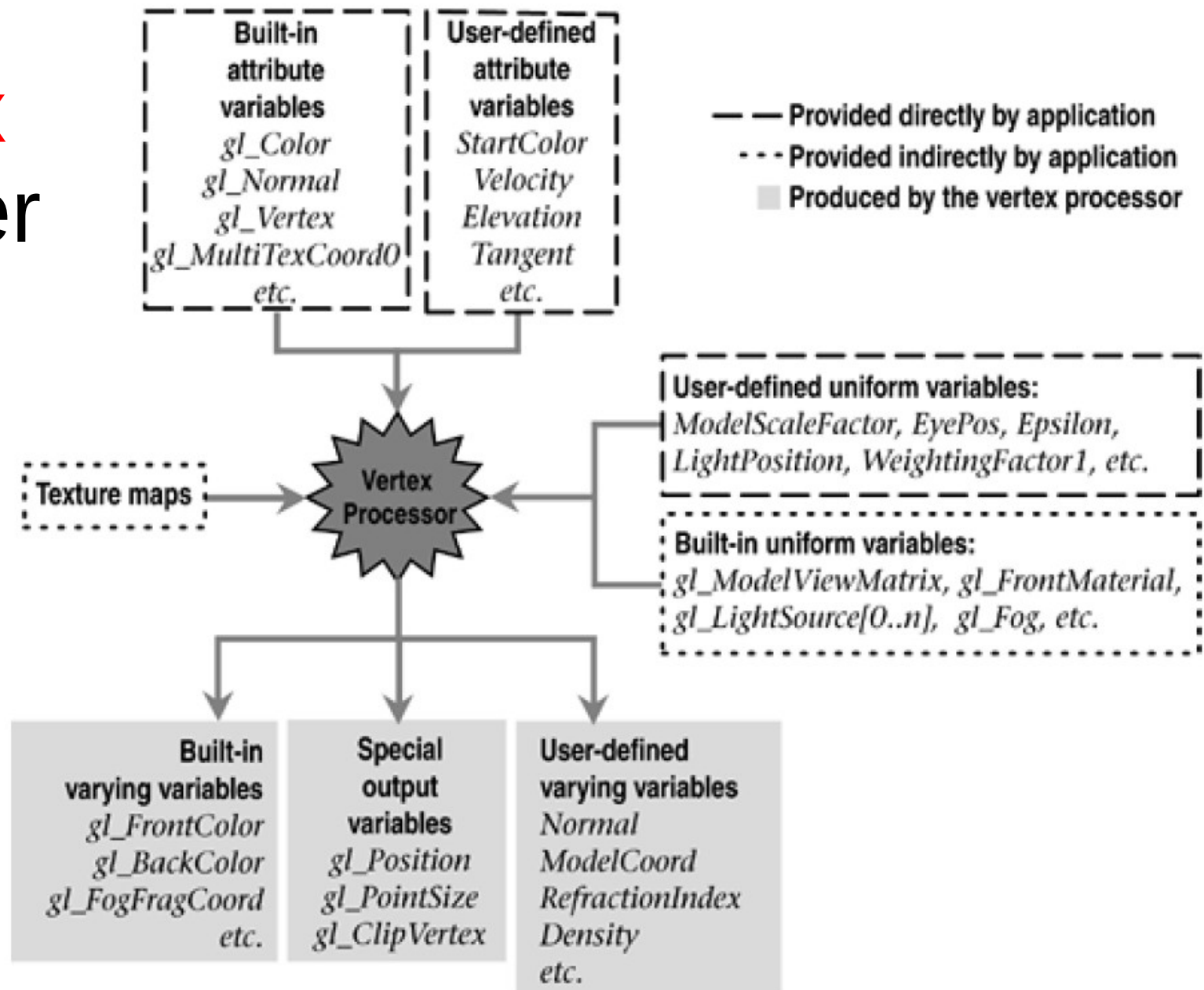
- **varying vec4 gl_TexCoord[];**

- **const**

- Para declarar dados somente de leitura : constantes como em C

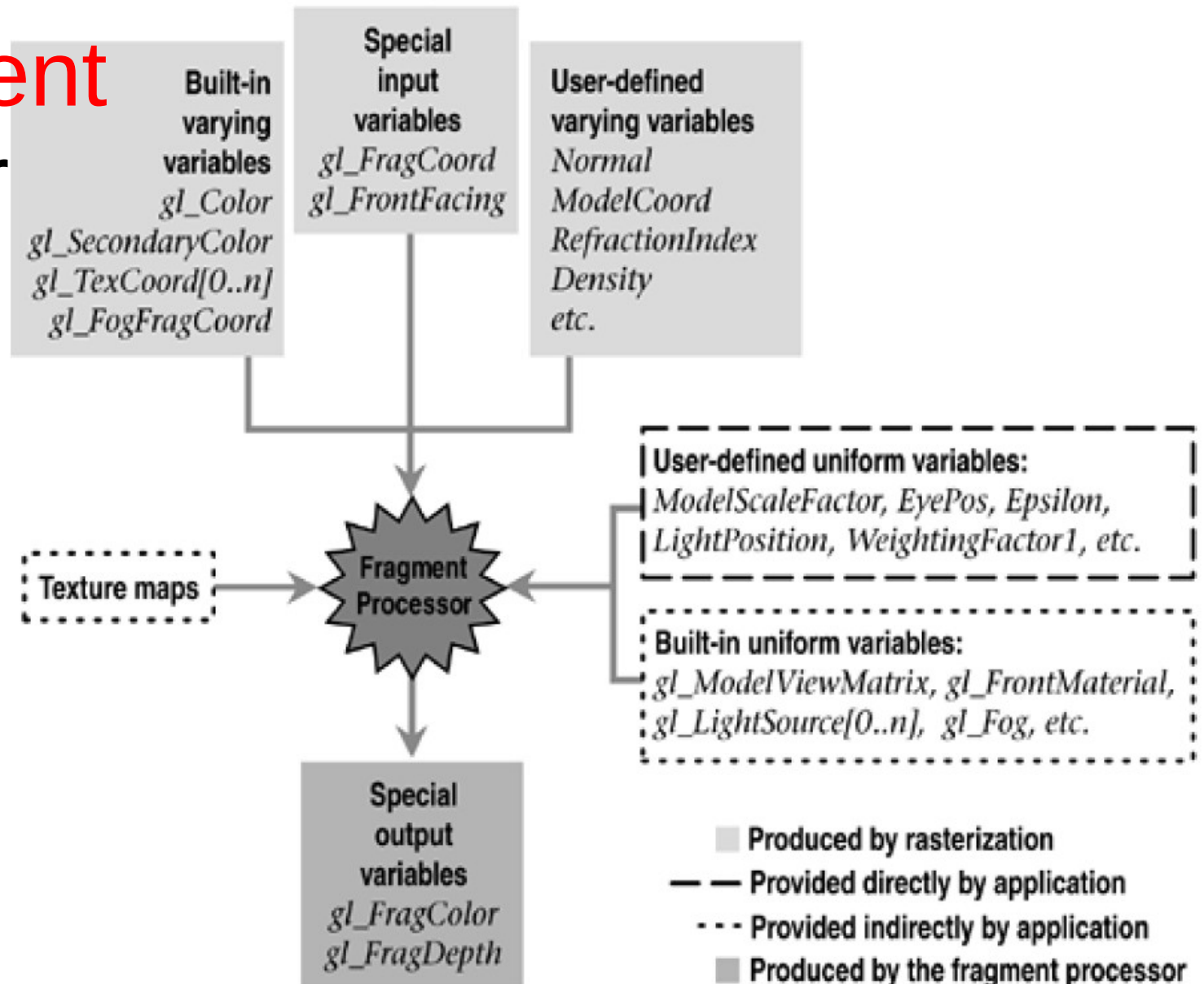
Variáveis disponíveis

- Vertex Shader



Variáveis disponíveis

- Fragment Shader



Exemplo Vertex Shader

```
#version 320 es
```

```
layout(location = 0) out vec4 position;
```

```
void main()  
{  
    gl_Position = position;  
};
```

Exemplo Fragment Shader

```
#version 320 es
```

```
precision mediump float;  
uniform float u_time;
```

```
layout(location = 0) out vec4 color;
```

```
void main()  
{  
    color = vec4(abs(sin(u_time)), 0.0, 0.0, 1.0);  
};
```

Dúvidas?



Atividade 1

- A partir dos códigos constantes nesses slides, crie um projeto chamado **GLFW_Shaders1**, faça sua compilação e execução de forma a poder verificar qual a figura que será desenhada e de que cor.

Atividade 1

- Não esqueça de incluir no início do programa, antes do main():

```
#include <GL/glew.h>
#include <GLFW/glfw3.h>
#include <iostream>

static void key_callback(GLFWwindow* window, int key, int scancode, int action, int mods)
{
    if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS) {
        glfwSetWindowShouldClose(window, GLFW_TRUE);
        std::cout << "Pressionado ESC : Saindo da aplicação!" << std::endl;
    }
}
```

Atividade 2

- Faça uma cópia do seu novo projeto para **GLFW_Pentagono** e o altere para que o mesmo desenhe um pentágono amarelo.

Atividade 3

- Como fazer para desenhar um quadrado azul sobreposto a outro verde, com desvio de 45° ?

